Ejercicio 1 - Más capas en el perceptrón multicapa (Iris)

Configuración del experimento

Se utilizó el conjunto Iris, compuesto por 150 muestras, cuatro atributos de entrada y tres clases. Las implementaciones conservaron la activación sigmoide, el error cuadrático, la tasa de aprendizaje de 0.03 y 500 épocas. Se comparó la red original **4 × 3 × 3** con una red profunda **4 × 3 × 3 × 3 × 3**, que incorpora dos capas ocultas adicionales.

Implementación	Topología	Error final	Comportamiento
NumPy	4 × 3 × 3	≈ 0.065	El error disminuye
NumPy	4 × 3 × 3 × 3 × 3	≈ 0.66	Descenso inicial y estancamiento
Keras	4 × 3 × 3	0.1475	Descenso gradual
Keras	4 × 3 × 3 × 3 × 3	0.2222	Estancamiento cerca de 0.2222

Los valores de NumPy se estimaron a partir de las gráficas. En Keras se tomaron los valores de loss registrados al finalizar la época 500. Las implementaciones calculan el error de manera diferente, de modo que sus magnitudes no deben compararse directamente.

Análisis comparativo

Al añadir dos capas ocultas, el error final **no mejoró** en ninguna implementación. En NumPy, la red original terminó aproximadamente en 0.065, mientras que la profunda se mantuvo cerca de 0.66. En Keras, la pérdida pasó de 0.1475 a 0.2222. Las curvas de las redes profundas muestran una reducción inicial y después se estabilizan, mientras que las redes originales mantienen un descenso más evidente.

Aunque las arquitecturas son equivalentes, las curvas de NumPy y Keras son distintas. El entrenamiento manual actualiza los pesos muestra por muestra según el orden de los datos, mientras que Keras utiliza lotes de 32 ejemplos y mezcla los datos de forma predeterminada. También existen diferencias en la inicialización de los pesos, el cálculo del error y la implementación de la retropropagación.

El estancamiento de las redes profundas es compatible con el **desvanecimiento del gradiente**. Al utilizar varias capas sigmoides, sus derivadas pueden reducir la señal utilizada para actualizar los pesos de las primeras capas. Por ello, agregar profundidad no garantiza mejores resultados en un conjunto pequeño como Iris. La predicción de Keras profundo, con valores próximos a 0.33 en las tres salidas, tampoco muestra una separación clara de las clases en el ejemplo evaluado.

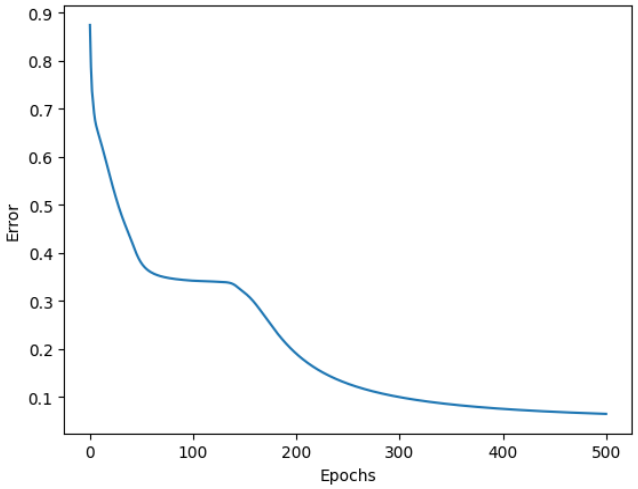
Las notebooks no registran tiempos totales de entrenamiento ni métricas de exactitud de las cuatro corridas; por ello no se incluye una comparación numérica de velocidad o precisión.

Curvas de entrenamiento

Las siguientes gráficas corresponden a las salidas de ejecución guardadas en las notebooks.

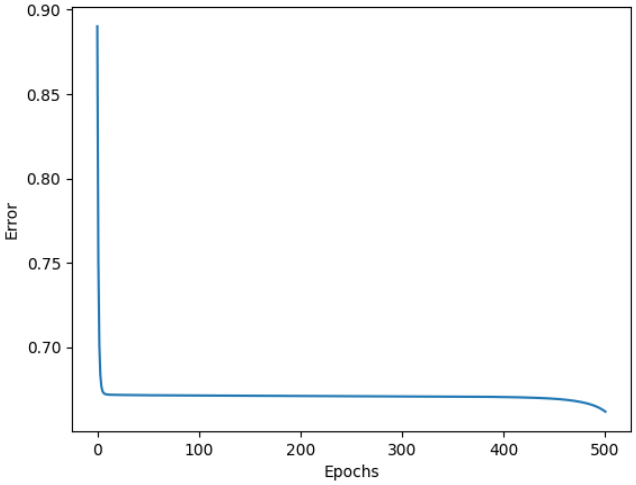
NumPy - red original

4 × 3 × 3 · error final ≈ 0.065



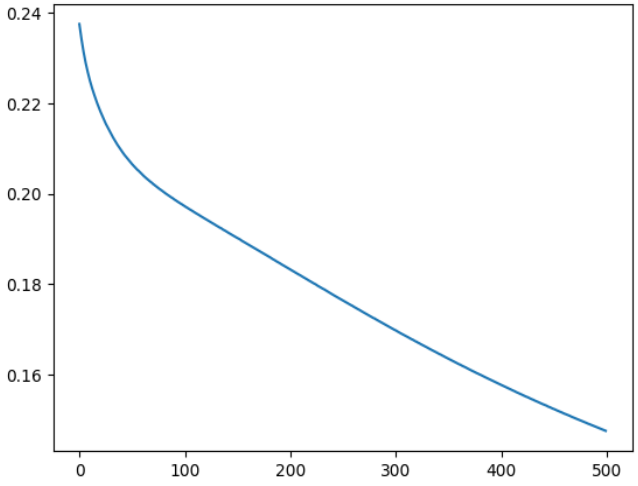
NumPy - red profunda

4 × 3 × 3 × 3 × 3 · error final ≈ 0.66



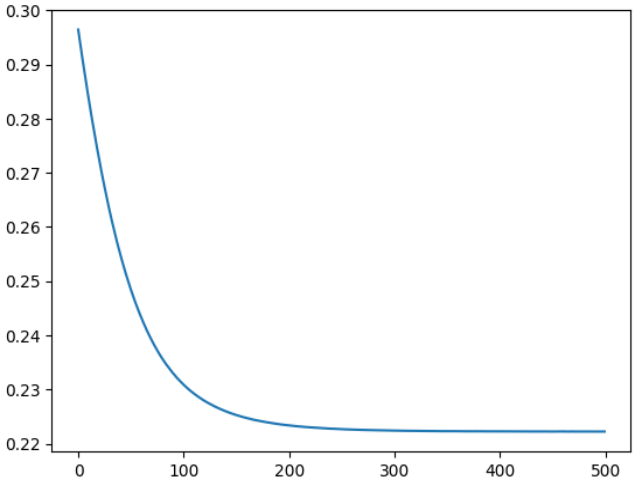
Keras - red original

4 × 3 × 3 · loss final = 0.1475



Keras - red profunda

4 × 3 × 3 × 3 × 3 · loss final = 0.2222



Las gráficas se obtuvieron de los resultados guardados en los archivos .ipynb; no corresponden a capturas adicionales del entorno de Google Colab.

Arquitecturas y predicciones de Keras

Resumen de los modelos

Red original: 4 × 3 × 3

Capa (tipo)	Forma de salida	Parámetros
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

Parámetros totales: 27 · Entrenables: 27 · No entrenables: 0

Red profunda: 4 × 3 × 3 × 3 × 3

Capa (tipo)	Forma de salida	Parámetros
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

Parámetros totales: 51 · Entrenables: 51 · No entrenables: 0

Predicción de ejemplo

Para la entrada **[3, 3, 1, 1]**, se registraron las siguientes salidas sigmoides. Estos valores no representan probabilidades normalizadas ni una medición de exactitud.

Red	Salida 1	Salida 2	Salida 3
Original	0.540865	0.335066	0.259652
Profunda	0.336871	0.333973	0.335643

Evidencia del entorno de ejecución

Las notebooks contienen salidas de ejecución y metadatos compatibles con Colab, pero no incluyen una captura del entorno o del menú Runtime. Esa evidencia queda pendiente de adjuntar.